Contents

Java interview guide, part 3: distributed systems, production, coding, and leadership

This is a guide to the interview, not just answers. Each question opens with what the interviewer is really testing and closes with the follow-ups they will ask next. Coworker-coaching voice throughout. This is **Part 3 of 3**.

Microservices

66. Authentication v/s authorization.

What they are testing: that you do not confuse the two, a quick but revealing check.

Answer. Authentication is "who are you" (proving identity, a login, a token, a certificate). **Authorization is "what are you allowed to do"** (checking permissions once identity is known). Authentication always comes first; authorization uses its result. A `401 Unauthorized` means you are not authenticated; a `403 Forbidden` means you are authenticated but not authorized.

Follow-ups. *401 v/s 403?* 401 = not (or wrongly) authenticated; 403 = authenticated but lacking permission. *Where does each happen in a microservices setup?* Authentication usually at the gateway/edge, authorization often per-service on the specific resource.

67. JWT v/s session.

What they are testing: the stateless-v/s-stateful trade-off and its consequences (especially revocation).

Answer. A **session** stores state on the **server** (the session id in a cookie maps to server-side data), so the server must look it up each request. A **JWT** is a **self-contained signed token** the client holds; the server just verifies the signature, no server-side lookup, so it is **stateless** and scales across services without a shared session store.

	Session	**JWT**
State	Server-side	Self-contained (client holds it)
Scaling	Needs shared session store	Stateless, no store
Revocation	Easy (delete server-side)	**Hard** (valid until expiry)
Size	Small id	Larger token on every request

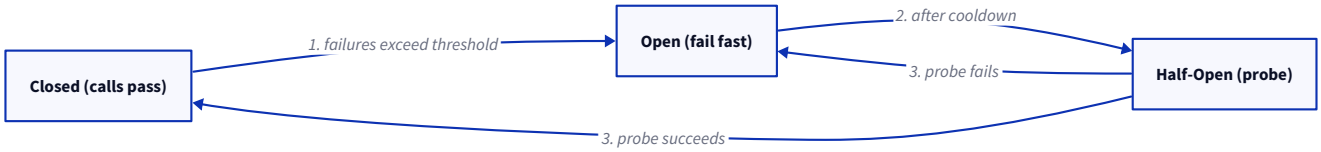
The senior point is **revocation**: you cannot easily invalidate a JWT before it expires, so you use **short expiry plus refresh tokens**, or a denylist, which claws back some statefulness.

Follow-ups. *How do you revoke a JWT?* Short TTL + refresh tokens, or a server-side denylist (giving up some statelessness). *Where do you store it client-side?* Not localStorage (XSS risk), prefer an httpOnly cookie.

68. Explain the circuit breaker pattern.

What they are testing: whether you know it prevents cascading failure, and the three states.

Answer. A circuit breaker **stops calling a failing dependency so you fail fast** instead of piling up doomed calls. It has three states: **Closed** (calls pass), **Open** (calls fail immediately without trying), and **Half-Open** (after a cooldown, a few probe calls test whether it recovered). It solves **cascading failure**: without it, a slow dependency ties up every caller's threads until the whole system stalls; the breaker cuts the call fast and lets you fall back.



Follow-ups. *Circuit breaker v/s retry?* Retry assumes a transient blip; the breaker assumes the dependency is down. Use them together, with the breaker outside the retry. *What triggers Open?* A failure-rate threshold over a window.

69. Explain retry.

What they are testing: whether you retry safely, not blindly.

Answer. Retry recovers from **transient** failures (a blip, a brief timeout). The rules that make it safe: use **exponential backoff with jitter** (not immediate, not in lockstep, or you get retry storms), cap the **attempts** (and use a retry budget), and only retry **idempotent** operations, retrying a non-idempotent call that may have already succeeded can double-charge. Never retry a client error (4xx) or a dependency that is clearly down.

Follow-ups. *Why jitter?* Without it, all clients retry at the same instant and hammer a recovering service. *Retry a POST?* Only if it is idempotent (an idempotency key makes it safe).

70. Explain timeout.

What they are testing: whether you know an unbounded wait is a system-killer; keep it short.

Answer. A timeout **bounds how long you wait** for a response, so a slow or dead dependency does not tie up your threads and connections indefinitely. Without one, a single hung call can exhaust a pool and take the whole service down. Set it from the dependency's normal latency plus headroom (around its p99), and make the **caller's timeout longer than the callee's** so the inner call fails first with a real error.

Follow-ups. *Too long v/s too short?* Too long ties up resources during an outage; too short kills healthy-but-slow requests. *Connection v/s read timeout?* One caps establishing the connection, the other caps waiting for data.

71. Explain idempotency.

What they are testing: whether you can make "at-least-once" delivery safe, essential for retries and messaging.

Answer. An operation is **idempotent if doing it twice has the same effect as doing it once**. It matters because networks give you at-least-once at best, so retries and redelivered messages **will** duplicate calls, and idempotency is what makes that safe. For a naturally non-idempotent action (charge a card, place an order), you use an **idempotency key**: the client sends a unique id, the server records it, and a repeat with the same key returns the original result instead of doing the work again. Among HTTP methods, GET/PUT/DELETE are idempotent, POST is not.

Follow-ups. *How do you implement it?* A unique key stored server-side (dedup table), checked before acting. *Why does messaging need it?* Kafka/queues are at-least-once, so consumers must handle duplicates.

72. Explain the API gateway.

What they are testing: whether you know why a single entry point helps in microservices.

Answer. An API gateway is a **single entry point in front of your services** that handles the **cross-cutting concerns** so each service does not: routing, **authentication**, **rate limiting**, TLS termination, request/response transformation, and sometimes **aggregation** (fan out to several services and combine). So clients talk to one endpoint, and the messy edge concerns live in one place instead of being duplicated per service.

Follow-ups. *Downside?* It is a potential single point of failure and a bottleneck, so it must be highly available. *Gateway v/s load balancer?* A load balancer just distributes traffic; a gateway adds routing, auth, and app-level logic.

73. Explain service discovery.

What they are testing: how services find each other when instances come and go (dynamic IPs).

Answer. In a dynamic environment, service instances start, stop, and move, so hard-coded addresses do not work. **Service discovery** is a **registry** where instances register themselves and callers look up healthy instances by name. Two styles: **client-side** (the caller queries the registry and picks an instance, like Eureka) and **server-side** (a load balancer or the platform does the lookup). In Kubernetes this is built in, a **Service** gives a stable DNS name that routes to the current healthy pods, so you rarely run a separate registry.

Follow-ups. *Client-side v/s server-side?* Client-side gives the caller control (and client load-balancing); server-side hides it behind a proxy. *In Kubernetes?* Service + DNS handles it natively.

74. Distributed transactions.

What they are testing: whether you know why classic 2PC is avoided across services and what replaces it.

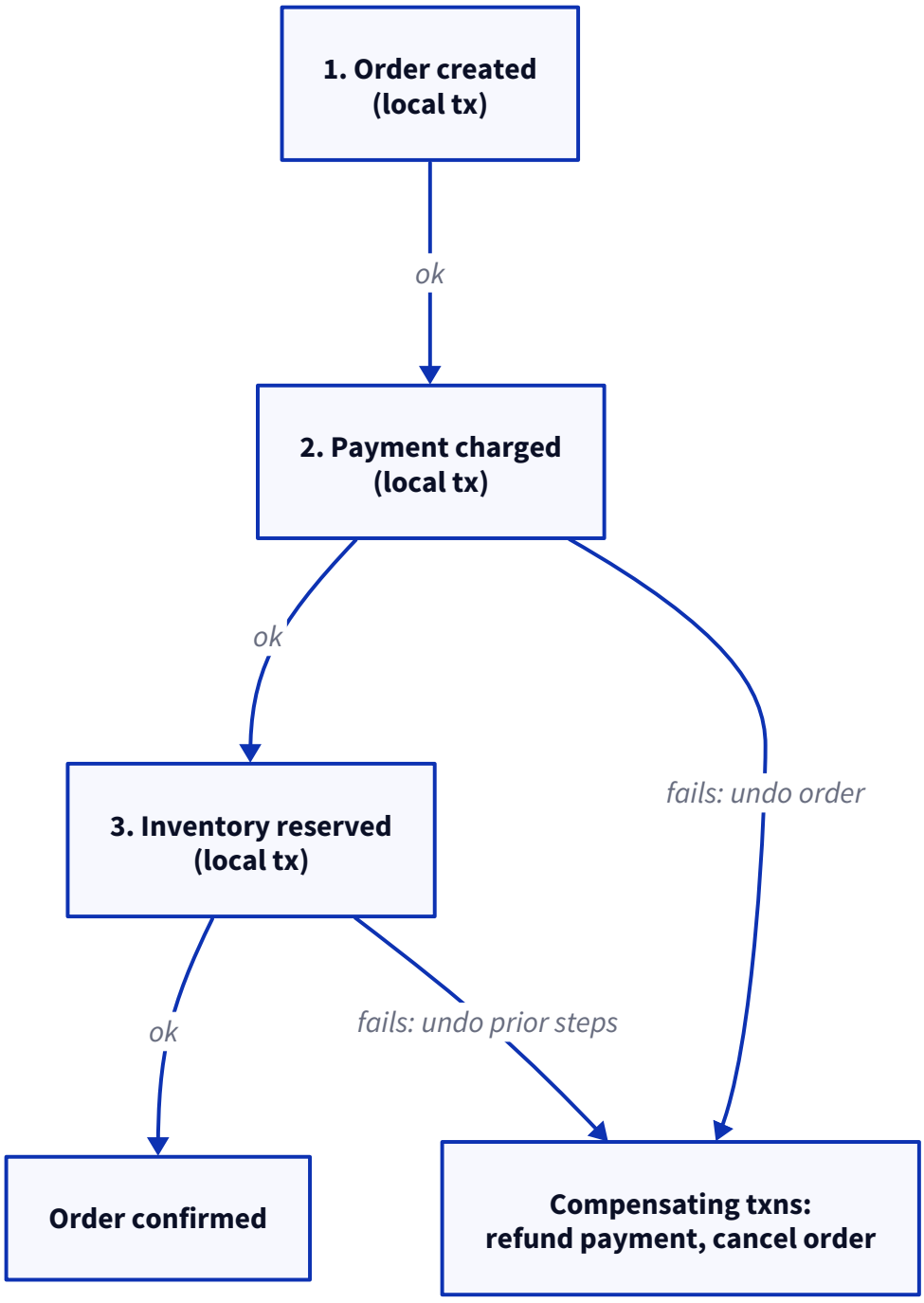
Answer. A transaction spanning multiple services/databases cannot use a normal ACID transaction. The classic answer, **two-phase commit (2PC)**, has a coordinator ask everyone to prepare then commit, but it **blocks resources while waiting, does not survive partitions well, and couples services tightly**, so it is avoided in microservices. The modern approach is the **saga** (next question): a sequence of local transactions with compensating actions, accepting **eventual consistency** instead of a global atomic commit.

Follow-ups. *Why not 2PC?* Blocking, poor partition tolerance, tight coupling, a slow/failed participant stalls everyone. *What do you give up with a saga?* Immediate consistency, you get eventual consistency and must design compensations.

75. Explain the saga pattern.

What they are testing: the go-to pattern for distributed data consistency; whether you know the two flavours and compensation.

Answer. A saga breaks a distributed transaction into a **sequence of local transactions**, one per service, where each step publishes an event that triggers the next. If a step fails, the saga runs **compensating transactions** to undo the previous steps (refund the payment, cancel the order), rather than a global rollback.



Two styles: **choreography** (each service listens for events and reacts, no central coordinator, simple but the flow is spread out) and **orchestration** (a central orchestrator tells each service what to do, clearer control and

easier to reason about, at the cost of a coordinator). You get **eventual consistency**, and each step must be idempotent (events can redeliver).

Follow-ups. *Choreography v/s orchestration?* Choreography is decentralised event reactions; orchestration has a central brain, better for complex flows. *What if a compensation fails?* You need retries and, ultimately, alerting/manual intervention, compensations must be robust.

76. Explain event-driven architecture.

What they are testing: whether you understand async decoupling and its consistency trade-off.

Answer. In event-driven architecture, services communicate by **producing and consuming events** (via Kafka, a queue) rather than calling each other directly. That **decouples** them (the producer does not know or wait for consumers), improves **resilience** (a down consumer just processes later) and **scalability** (add consumers independently). The trade-off is **eventual consistency** and harder reasoning: flows are asynchronous, you need idempotent consumers (at-least-once delivery), and debugging spans multiple services, so you lean on tracing and correlation ids.

Follow-ups. *Downside?* Eventual consistency, harder debugging, duplicate handling. *How do you trace a flow?* Propagate a correlation/trace id through the events. *Delivery guarantee?* Usually at-least-once, so consumers must be idempotent.

Production

These are "walk me through it" questions. The interviewer is testing your **method under pressure**, not a memorised cause. The winning shape is always: **observe (metrics) to isolate (which resource / which dependency) to drill (traces, dumps, logs) to fix**, and to say what you would rule out. Answer from that structure.

77. A service suddenly becomes slow. How do you investigate?

Answer. I start at the **dashboards**, not the code, to localise it: is **latency** up everywhere or on one endpoint, and did it start at a **deploy** (check the deploy marker) or a traffic spike? Then I isolate the **resource**: CPU, memory/GC, thread pool, or a **downstream** (database, another service, cache). **Distributed traces** show which hop got slow, and **logs** on that hop give the specifics. The usual culprits are a **slow dependency**, **GC pauses**, **lock contention**, or a bad query. So: localise from metrics, isolate the resource or hop with traces, confirm in logs, and check whether a deploy lines up.

Follow-ups. *First thing you look at?* Whether it correlates with a deploy, that is the fastest signal. *Slow for everyone v/s one endpoint?* One endpoint points at that code/query; everything points at a shared resource (CPU, GC, DB pool).

78. CPU reaches 100%.

Answer. Find the **hot thread**, then read its stack. `top -H` finds the thread burning CPU, then a **thread dump** (`jstack`) shows exactly what that thread is running, usually a **tight loop**, or **excessive GC** (check GC time, high GC often masquerades as high CPU). So it is: hot thread, its stack, the code. If it is GC, the fix is heap/collector tuning; if it is a loop, it is a code bug.

Follow-ups. *High CPU but low throughput?* Likely GC thrashing. *How do you map the OS thread to Java?* The thread's `nid` (hex) matches the thread dump.

79. Memory keeps increasing.

Answer. This is a **leak** until proven otherwise: after each **full GC**, does used heap keep climbing instead of returning to a baseline? If yes, take a **heap dump** and use the **dominator tree** to find the biggest retainer, then trace the reference chain to the root. The usual suspects: a **growing cache/collection with no eviction**, **ThreadLocals not cleaned up in a pool**, or **unclosed resources**. Fix the retention.

Follow-ups. *Leak v/s just needs more heap?* A leak grows without bound; a small heap plateaus. *Tool?* Heap dump + Eclipse MAT, or JFR over time.

80. Kubernetes keeps restarting pods.

Answer. I check **why** with `kubectl describe pod` and the previous container's logs. The common causes: **OOMKilled** (the container hit its memory **limit**, often because the JVM heap `-Xmx` was set higher than the container limit, so tune heap to the limit or raise the limit), a **failing liveness probe** (the app is slow to start or unhealthy, so k8s kills it, fix the probe timing or the health), or a **crash on startup** (read the logs). So: `describe` for the reason (OOMKilled v/s probe v/s crash), then fix that specific cause.

Follow-ups. *OOMKilled but heap looks fine?* The JVM uses more than heap (Metaspace, threads, direct buffers); size the container for total footprint, not just `-Xmx`. *CrashLoopBackOff?* The container keeps failing to start, the logs and `describe` tell you why.

81. High GC pauses.

Answer. First, **which collector and how bad** (GC logs: pause times and frequency). Long pauses usually mean the **heap is too small** (frequent full GCs), **too much garbage** being allocated (churn), or a collector mismatch. Fixes, in order: **reduce allocation** on the hot path (the real fix), **size the heap** appropriately, and for latency-sensitive apps **use a low-pause collector** (G1 by default, or ZGC/Shenandoah for sub-millisecond pauses on large heaps). So: read GC logs, cut allocation, right-size the heap, and pick the right collector.

Follow-ups. *G1 v/s ZGC?* G1 balances throughput and pauses; ZGC targets very low pause on large heaps. *Biggest lever?* Usually reducing allocation, not tuning flags.

82. Thread pool exhaustion.

Answer. All pool threads are **busy or blocked**, so new work queues or is rejected. A **thread dump** shows what they are stuck on, almost always **blocked on a slow downstream call** (no timeout) or **lock contention**. The fixes: add **timeouts** on every external call (so threads are not held forever), a **circuit breaker** so a down dependency fails fast instead of consuming the pool, **bulkheads** (separate pools per dependency) so one slow dependency cannot starve the rest, and right-size the pool. So the root cause is usually a missing timeout on a downstream, not the pool size itself.

Follow-ups. *Just make the pool bigger?* Rarely the fix, if threads block forever, a bigger pool just delays exhaustion. *Bulkhead?* Isolate each dependency in its own pool so one failure is contained.

83. Redis is down.

Answer. The key principle: a **cache outage should degrade, not kill, the app**. If Redis is a cache, the service should **fall back to the source of truth** (the database) rather than failing, with a **circuit breaker** so it fails fast on Redis instead of every request timing out, and a **timeout** on Redis calls. The risk to name is a **thundering herd**: if the cache is empty, every request hits the database at once, so you protect the DB (request coalescing,

limited concurrency). If Redis is used as a **datastore** (not just cache), that is different, then it is a real outage and you rely on its HA/failover.

Follow-ups. *Cache v/s datastore?* A cache miss can fall back to the DB; a datastore outage is a hard dependency failure. *Thundering herd?* Coalesce concurrent misses, or serve slightly stale data.

84. Kafka consumer lag increases.

Answer. Lag means **consumers are falling behind producers**. I check whether processing is **slow** (each message takes too long, an external call or DB write in the consumer) or whether **throughput is capped** by partitions (you cannot have more consumers than partitions in a group). Fixes: **speed up processing** (batch, async the slow part), **add consumers up to the partition count** (and add partitions if you need more parallelism), and check for **frequent rebalances** (which stall consumption) or a **poison message** blocking progress. So: is it slow processing or partition-capped, then scale the right one.

Follow-ups. *More consumers than partitions?* The extras sit idle, partitions cap parallelism. *A single bad message stalling it?* Handle with a dead-letter topic so one message does not block the partition.

85. Database connection pool is exhausted.

Answer. Connections are being **held too long or not returned**. The causes: **long-running queries or transactions** holding connections, a **connection leak** (code that does not close/return them), or the **pool is simply too small** for the load. I check the pool metrics and slow-query logs, and take a thread dump to see threads **waiting for a connection**. Fixes: **fix the slow queries / shorten transactions** (do not hold a connection across a slow external call), **ensure connections are always returned** (try-with-resources, or the framework managing it), add **acquisition timeouts**, and size the pool sensibly. Usually it is a leak or a long transaction, not just a small pool.

Follow-ups. *Bigger pool?* Only if the DB can handle it; often the real fix is releasing connections faster. *Connection held across an HTTP call?* Classic mistake, never hold a DB connection while waiting on a remote call.

Coding

For these, the interviewer watches **how you think, not just whether it compiles**: do you pick the right data structure, do you say the complexity out loud, do you handle thread safety and edge cases, and can you talk about trade-offs. Start by clarifying constraints (size, concurrency, thread-safety), state the approach and its Big-O, then code.

86. LRU cache.

What they are testing: the key insight that you need **O(1) get and put**, which forces a **HashMap plus a doubly-linked list** (the map finds the node, the list tracks recency). They usually want it from scratch.

Answer. Map key to a list node for O(1) lookup; keep a doubly-linked list ordered by recency (most-recent at the head). On access, move the node to the head; on insert past capacity, evict the tail (least recent).

```java
class LRUCache {
    private final int capacity;
    private final Map<Integer, Node> map = new HashMap<>();
    private final Node head, tail;                  // dummy sentinels

    LRUCache(int capacity) {
        this.capacity = capacity;
        head = new Node(0, 0); tail = new Node(0, 0);
        head.next = tail; tail.prev = head;
    }
    public int get(int key) {
        Node n = map.get(key);
        if (n == null) return -1;
        moveToFront(n);                             // mark most-recently-used
        return n.value;
    }
    public void put(int key, int value) {
        Node n = map.get(key);
        if (n != null) { n.value = value; moveToFront(n); return; }
        if (map.size() == capacity) {               // evict least-recently-used (tail.prev)
            Node lru = tail.prev; remove(lru); map.remove(lru.key);
        }
        Node fresh = new Node(key, value);
        map.put(key, fresh); addFront(fresh);
    }
    private void moveToFront(Node n) { remove(n); addFront(n); }
    private void addFront(Node n) { n.next = head.next; n.prev = head; head.next.prev = n;
        head.next = n; }
    private void remove(Node n) { n.prev.next = n.next; n.next.prev = n.prev; }

    static class Node { int key, value; Node prev, next; Node(int k, int v){ key=k; value=v;
        } }
}
```

The shortcut worth mentioning: `LinkedHashMap` with `accessOrder=true` and an overridden `removeEldestEntry` gives you an LRU in a few lines, good to say you know it, but they usually want the from-scratch version to see the map-plus-DLL insight.

Follow-ups. *Make it thread-safe?* Wrap operations in a lock, or use a concurrent design; a plain `HashMap` here is not thread-safe. *Why a doubly-linked list?* O(1) removal from the middle (a singly-linked list would be O(n) to find the previous node). *LFU instead?* Different structure, track frequencies (a frequency map of lists).

87. Producer-Consumer.

What they are testing: whether you reach for the right concurrency tool (`BlockingQueue`) rather than hand-rolling `wait` / `notify` , and whether you can do both.

Answer. The idiomatic solution is a **`BlockingQueue`** , which handles all the waiting and signalling for you: `put` blocks when full, `take` blocks when empty.

```
BlockingQueue<Task> queue = new ArrayBlockingQueue<>(100);   // bounded

// producer thread
queue.put(task);      // blocks if the queue is full

// consumer thread
Task t = queue.take(); // blocks if the queue is empty
```

I would say upfront that I would use a `BlockingQueue` in real code, and only hand-roll it with `wait` / `notify` if the interviewer explicitly wants the low-level version (see the custom blocking queue below). The bounded queue also gives you **backpressure**, a slow consumer naturally slows the producer.

Follow-ups. *Why bounded?* Backpressure and memory safety, an unbounded queue lets a fast producer exhaust memory. *Multiple producers/consumers?* `BlockingQueue` is thread-safe, so it works unchanged. *Graceful shutdown?* A poison-pill sentinel or interrupt the consumers.

88. Thread-safe singleton.

What they are testing: whether you know the two clean options and avoid the fiddly ones.

Answer. Two solid choices. The **enum** is the simplest and is immune to reflection and serialization attacks. The **Bill Pugh holder idiom** is the best lazy class-based option, thread-safe via class-loading, no locks, no `volatile`.

```
// Option A: enum — thread-safe, reflection- and serialization-proof, eager
enum Config { INSTANCE; /* fields and methods here */ }

// Option B: Bill Pugh holder — lazy, thread-safe, no locking
class Config {
    private Config() {}
    private static class Holder { static final Config INSTANCE = new Config(); }  // loaded
        on first use
    public static Config getInstance() { return Holder.INSTANCE; }
}
```

I would avoid the `synchronized`-method version (locks on every call) and hand-rolled double-checked locking (the `volatile` is easy to forget).

Follow-ups. *Why does the holder idiom work?* The JVM loads the inner class only when `getInstance()` first references it, and class initialisation is thread-safe by spec. *Enum downside?* Eager and cannot extend a class.

89. Rate limiter.

What they are testing: whether you know a real algorithm (token bucket or sliding window) and can implement it thread-safely.

Answer. The **token bucket** is the common choice: tokens refill at a fixed rate up to a capacity, each request takes a token, and no token means rejected. It allows short bursts (up to the bucket size) while capping the sustained rate.

```java
class TokenBucket {
    private final long capacity;
    private final double refillPerSec;
    private double tokens;
    private long lastRefill = System.nanoTime();

    TokenBucket(long capacity, double refillPerSec) {
        this.capacity = capacity; this.refillPerSec = refillPerSec; this.tokens = capacity;
    }
    public synchronized boolean tryAcquire() {
        refill();                                          // credit tokens for elapsed time
        if (tokens >= 1) { tokens -= 1; return true; }  // allowed
        return false;                                      // rate limit hit -> reject
    }
    private void refill() {
        long now = System.nanoTime();
        double elapsedSec = (now - lastRefill) / 1e9;
        tokens = Math.min(capacity, tokens + elapsedSec * refillPerSec);
        lastRefill = now;
    }
}
```

Follow-ups. *Token bucket v/s sliding window?* Token bucket allows bursts; a sliding-window log is more precise but heavier. *Distributed rate limiting?* Move the counter to Redis (atomic `INCR` with expiry, or a Lua script) so all instances share it. *Fixed window problem?* Bursts at the window boundary can allow 2x the limit briefly.

90. Custom blocking queue.

What they are testing: the raw `wait` / `notify` version, whether you use `while` (not `if`), hold the monitor correctly, and use `notifyAll`.

Answer. A bounded queue guarded by the intrinsic lock: `put` waits while full, `take` waits while empty, and both re-check the condition in a **`while` loop** (for spurious wakeups) and `notifyAll` after changing state.

```java
class BoundedBlockingQueue<T> {
    private final Queue<T> queue = new LinkedList<>();
    private final int capacity;
    BoundedBlockingQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (queue.size() == capacity) wait();   // wait for space (while, not if)
        queue.add(item);
        notifyAll();                               // signal waiting consumers
    }
    public synchronized T take() throws InterruptedException {
        while (queue.isEmpty()) wait();            // wait for an item
        T item = queue.remove();
        notifyAll();                               // signal waiting producers
        return item;
    }
}
```

The three things they check: **`while` not `if`** (spurious wakeups), calling `wait` / `notify` **inside `synchronized`** (you must hold the monitor), and **`notifyAll` not `notify`** (producers and consumers wait on

the same lock, so `notify` might wake the wrong side).

Follow-ups. *Why `notifyAll`?* `notify` could wake another producer when a consumer needed waking, deadlocking progress. *`Condition` instead?* A `ReentrantLock` with separate `notFull` / `notEmpty` conditions lets you signal exactly the right waiters, which is what `ArrayBlockingQueue` does.

91. Design a cache.

What they are testing: breadth of design thinking, this is open-ended, so lead with the decisions, not code.

Answer. I would walk through the dimensions rather than jump to code. **Eviction policy**: LRU (recency), LFU (frequency), FIFO, or TTL (time-based), picked from the access pattern. **Size bound**: cap by entries or memory, with eviction when full. **Thread safety**: a `ConcurrentHashMap` for the store, or striped locks, so it scales under concurrent access. **Write policy**: write-through (write cache and DB together, consistent) v/s write-back (write cache, flush later, faster but riskier). **Expiry**: TTL for freshness. **Local v/s distributed**: an in-process map (fast, per-instance) v/s a shared store like **Redis** (consistent across instances). And **stampede protection**: when a hot key expires, coalesce the concurrent misses so they do not all hit the database at once. In real life I would reach for **Caffeine** (local) or **Redis** (distributed) rather than build it, and I would say so.

Follow-ups. *LRU v/s LFU?* LRU suits recency-biased access; LFU keeps genuinely popular items but adapts slower. *How do you keep a distributed cache consistent with the DB?* TTLs, or invalidate on write (cache-aside). *Cache stampede fix?* Request coalescing, or slightly-early async refresh.

92. Design a logger.

What they are testing: a clean, extensible design and awareness of the performance angle (async logging), open-ended.

Answer. The core pieces: **log levels** (TRACE/DEBUG/INFO/WARN/ERROR) with a configurable threshold so you filter cheaply; a **`Logger` facade** the app calls; **appenders** (console, file, network) so the destination is pluggable (Strategy pattern); and **formatters** for the message layout. The two senior points: it must be **thread-safe** (many threads log at once), and for performance you make it **asynchronous**, the logging call just **enqueues** the message and a **background thread** writes it, so application threads are not blocked on disk or network I/O.

```
enum Level { TRACE, DEBUG, INFO, WARN, ERROR }

interface Appender { void append(Level level, String message); }  // console, file, ...

// Logger checks the threshold, formats, and hands off to appenders
// (async variant: put the record on a queue; a background thread drains it)
```

Follow-ups. *Why async?* So a slow disk or network appender does not block request threads; the trade-off is you can lose the tail of the buffer on a hard crash. *How do you not lose logs on crash?* Flush on shutdown, or accept bounded loss. *This is basically Log4j/SLF4J?* Yes, and in practice I would use SLF4J with Logback rather than build it.

93. Design a parking lot (LLD).

What they are testing: object-oriented modelling, whether you find the right classes and responsibilities and keep them cohesive. This is a low-level-design round, so give the class model, not a full program.

Answer. I would model it as a small set of cohesive classes:

- `ParkingLot` : the top-level system, holds levels, and exposes `parkVehicle` / `unpark` .
- `Level` : a floor, holds its parking spots and tracks availability.
- `ParkingSpot` : a single spot with a **size** (compact, large, handicapped, EV) and an occupied flag.
- `Vehicle` (abstract) with subtypes `Car` / `Bike` / `Truck` , each mapping to the spot sizes it can use.
- `Ticket` : issued on entry (spot, entry time), used to compute the fee on exit.
- `ParkingStrategy` : how to pick a spot (nearest, first-available), kept separate so it can change without touching the lot (Strategy pattern).
- `FeeCalculator` : pricing by duration/vehicle type, also separate so pricing rules can vary.

The design points an interviewer probes: **how you match a vehicle to a compatible spot size**, **concurrency** (two cars racing for the last spot, so spot assignment must be atomic/synchronised), and **extensibility** (adding an EV spot or a new pricing rule should not require rewriting the core, which is why strategy and fee calculation are pulled out).

Follow-ups. *How do you handle the last-spot race?* Synchronise or use an atomic reservation so two entries cannot claim the same spot. *Add EV charging spots?* A new spot size/subtype and a strategy tweak, no core rewrite (Open/Closed). *Find-availability at scale?* Keep per-level free-spot counts so you do not scan every spot.

Leadership and tech lead

These are behavioural, and the bar is different: the interviewer is not checking whether you *can* code, they are checking whether you can be **trusted with people, quality, and judgment**. Vague or humble-brag answers sink here. Answer with a clear philosophy and a concrete example, and show you think beyond your own keyboard. All of these are spoken from experience, so adapt them to your own stories.

94. How do you review pull requests?

What they are testing: whether you review for the things that matter and give feedback that improves both the code and the person, rather than nitpicking style or rubber-stamping.

Answer. My priority order is **correctness first, then design and maintainability, then style last** (and I let a formatter and linter own style so humans never argue about it). Before commenting I make sure I **understand the intent** of the change, reviewing without context leads to bad feedback. I check that **tests exist and actually cover the change**, not just that they pass. When I comment I am **specific and explain the why**, and I clearly separate **blocking issues from suggestions** (I prefix the optional ones with "nit:") so the author knows what must change. And I keep the loop **fast**: a quick approve on small low-risk changes, a deeper pass on the risky ones. The goal is better code and a better engineer, not showing I found something.

Follow-ups. *A huge PR?* Ask to split it, you cannot review 2,000 lines well; if you must, review in logical chunks. *Disagreement with the author?* Discuss the trade-off, and if it is a genuine judgment call and not a real problem, defer, it is their code. *Avoiding bikeshedding?* Automate style so the review is about substance.

95. What coding standards do you enforce?

What they are testing: whether you have a pragmatic, automated bar, not personal dogma imposed on everyone.

Answer. The principle is **automate everything a machine can check** (formatting, linting, static analysis, coverage gates in CI) so human review is spent on **substance, not spaces**. Beyond that, the standards I care

about are the ones that affect **readability and safety**: meaningful names, small focused functions, **tests required** for new logic, proper **error handling** (no swallowed exceptions), no commented-out or dead code, and no secrets in the repo. The key is that standards are **written down and agreed by the team**, not invented per review, and **consistency matters more than my personal preference**, a consistent codebase is easier for everyone. I enforce them through CI (which is objective) plus review for the judgment calls.

Follow-ups. *What do you deliberately not enforce?* Style the formatter already owns, arguing about it is waste. *Someone disagrees with a standard?* Raise it with the team and change the written standard, do not litigate it in every PR. *New standard on a legacy codebase?* Apply going forward, do not block work on a big-bang cleanup.

96. How do you mentor junior engineers?

What they are testing: whether you can grow people (a force multiplier for the team), with patience and the ability to let go.

Answer. The way I approach it: give them **real, meaningful work but scoped with a safety net**, ownership is how people grow, but not by being thrown in the deep end. I **pair and review with explanation**, always the *why* behind a change, not just the fix, so they learn the reasoning. I try to **ask questions rather than hand over answers** ("what happens if this is null?") so they build the instinct themselves, and I let them **struggle productively** for a bit before stepping in, because that is where learning sticks. Then I **gradually increase autonomy** as they earn it, and I make a point of recognising progress. The measure of good mentoring is that they need me less over time.

Follow-ups. *Mentoring v/s your own delivery?* Time spent mentoring pays back in team throughput, but I protect some focus time; it is a balance, not either/or. *A struggling mentee?* Get specific about the gap, pair more closely, give smaller wins to rebuild confidence. *When do you step in v/s let them fail?* Let them fail on reversible, low-stakes things; step in before anything hits production or a hard deadline.

97. Tell me about a production incident you handled.

What they are testing: ownership, calm under pressure, a structured response, and, crucially, the **follow-through** so it does not happen again. They want to see mitigate-then-fix, not heroics.

Answer. I would tell it in the order it actually happened, and the shape matters more than the specifics. **Detection**: an alert fired (an SLO burn or error-rate spike), not a customer report. **Mitigate first**: before hunting the root cause, I stop the bleeding, roll back the suspect deploy, fail over, or degrade the feature, because restoring service beats understanding it in the moment. **Communicate**: post status so stakeholders are not guessing. **Then root cause**: with the pressure off, use the traces and logs to find the real cause. **Then a blameless postmortem**: write up the timeline and, most importantly, **action items that prevent recurrence** (a new monitor, a missing timeout, a guardrail), and follow up that they get done. A representative one: publish failures spiked for a single downstream provider, the SLO alert caught it, I failed the calls over to a fallback to mitigate, confirmed it was the provider (no deploy correlated), and afterwards added a per-provider circuit breaker and monitor. *(Tell your own real incident; keep the detect-mitigate-communicate-fix-prevent shape.)*

Follow-ups. *Mitigate or root-cause first?* Mitigate, always, users care about service, not your diagnosis. *Why blameless?* Blame hides information; you want people to surface what really happened. *What did you change to prevent it?* Name the concrete guardrail, this is the part that shows seniority.

98. If you were the tech lead, what would you change in this architecture?

What they are testing: architectural judgment and prioritisation, and diplomacy, because you are often critiquing a system the interviewers built. They want high-impact thinking without arrogance or "rewrite it all".

Answer. First I would **understand why it is built this way** before proposing anything, there are usually constraints and history behind decisions that look wrong from outside, so I ask before I judge. Then I would **prioritise by impact and risk, not personal taste**: reliability and the biggest real pain points first. Concretely, the changes that most often pay off, if they are missing: **observability and SLOs** (you cannot improve what you cannot see), **resilience on cross-service calls** (timeouts, circuit breakers), **breaking a painful tight coupling** between services, and **a faster, safer test-and-deploy pipeline**. I would frame all of it as **incremental steps, not a rewrite**, because big rewrites usually fail, and I would **propose and discuss rather than dictate**. So: understand the context, target the highest-impact reliability wins, and change it incrementally with the team, not around them.

Follow-ups. *Rewrite v/s incremental?* Almost always incremental; a rewrite throws away hard-won knowledge and risks a long feature freeze. *How do you propose big changes without alienating the team?* Ask questions, bring data (incidents, metrics), pilot on one service, and give credit. *How do you prioritise tech debt against features?* Frame debt in terms of the risk and velocity cost it creates, and tie it to business impact so it competes fairly.

That is the full guide, across all three parts, from the interviewer's side of the table. The pattern for every question is the same: work out what is really being tested, lead with the answer, go as deep as that question warrants, and be ready for the follow-up.